

OSC CW2025

Group 1

Student Name	Student ID	Course Name
Ibuki Furusho	20716304	Computer Science with AI
Kazuki Ichikawa	20605542	Computer Science with AI
Jeffery Teoh Dass		Computer Science with AI

Table of Content

1. Implementation Details	3
1.1 Process Control Block (PCB) Design	3
1.2 SRTF Scheduling Algorithm Logic	4
1.3 Multithreading and Synchronization Strategy	4
1.4 State Management	4
1.5 Input Validation and Robustness	5
2. Test Cases	5
Test Case 1 : Preemption occurs frequently	5
Test Case 2 : CPU IDLE	6
Test Case 3 : Long waiting queue	9
Test case 4 : Simultaneous Arrival	11
Test Case 5 : Confirming long job starvation	13
Input Validation	15
3. Discussion	15
4.1 Quantitative Comparison between Theory and Simulation	15
4.2 Impact of Process Arrival Variation on Efficiency	15
4. Conclusion	16
5. Appendix Source code listing (with comments for readability)	17
6. AI Tools and External Sources Usage	17
7. Peer Review form	17

Any supporting explanations about your code or technique used in your work
Answering questions (refer the numbers 1-6 in the report part of the question) with 4 sets of
Input/output screen displays

1. Implementation Details

This section outlines the architectural design and technical strategies employed to simulate the Shortest Remaining Time First (SRTF) scheduling algorithm using the C language. The implementation leverages multithreading to accurately model concurrent process execution and synchronization.

Build & Execution

Linux / WSL

Bash

```
gcc -std=c17 -Wall -Wextra -g MultiThreading.c -o srtf -lpthread  
./srtf
```

Windows (MinGW-w64 / MSYS2 / Git Bash)

Install the MinGW-w64 toolchain, then run:

Bash

```
gcc -std=c17 -Wall -Wextra -g MultiThreading.c -o srtf.exe -lpthread  
./srtf.exe
```

If you prefer MSVC, compile with:

Bash

`cl /std:c17 /W4 /Zi MultiThreading.c /Fe:srtf.exe and link against a pthreads port such as pthreadVC2.lib.`

macOS (Homebrew clang)

Bash

```
brew install llvm
```

```
/opt/homebrew/opt/llvm/bin/clang -std=c17 -Wall -Wextra -g MultiThreading.c -o srtf -lpthread  
./srtf
```

Add [/opt/homebrew/opt/llvm/bin](#) to your [PATH](#) or call the Xcode clang if pthread support is available.

1.1 Process Control Block (PCB) Design

To represent processes within the system, a `Process` structure was defined, acting as a Process Control Block (PCB). This structure encapsulates all necessary attributes for scheduling and threading.

- **Scheduling Attributes:** Includes ID, ArrivalTime, BurstTime, and RemainingBurstTime. The RemainingBurstTime is critical for SRTF, as it is dynamically updated at each time unit.
- **Performance Metrics:** Stores CompletionTime, TurnaroundTime, WaitingTime, and ResponseTime for post-simulation analysis.
- **Threading Primitives:** Contains a `pthread_t` identifier and a unique `pthread_cond_t` (condition variable) to control the execution flow of each specific process thread.

Code Excerpt: Process Structure

```
...
```

```

typedef struct {
    int ID;
    int ArrivalTime;
    int BurstTime;
    int RemainingBurstTime;
    int CompletionTime;
    process_state_t status; // Enum: STATE_READY,
                           STATE_RUNNING, STATE_COMPLETED
    int TurnaroundTime;
    int WaitingTime;
    int ResponseTime;

    // Threading resources
    pthread_t thread;
    pthread_cond_t my_cond;
    int run_permission; // 1 = allowed to run, 0 = wait

    // Reference to context
    SimulationContext *ctx;
} Process;

```

...

Each 'Process' embeds a pointer to the shared 'SimulationContext'. This context encapsulates the mutex, scheduler condition variable, log buffer, and termination flags, ensuring threads do not rely on global variables. This architecture promotes modularity and enables thread safety by restricting state access to the context pointer. Furthermore, process states are strictly defined using the 'process_state_t' enum ('STATE_READY' / 'STATE_RUNNING' / 'STATE_COMPLETED'), ensuring type-safe state transitions and improving code readability.

1.2 SRTF Scheduling Algorithm Logic

The core scheduling logic is encapsulated in the `search_shortest` function. The scheduler operates in discrete time steps ($t, t+1, \dots$) and performs the following checks at every unit of time:

1. **Filter:** Identifies processes that have arrived ($\text{ArrivalTime} \leq \text{current_time}$) and are not yet completed ($\text{RemainingBurstTime} > 0$).
2. **Selection:** Iterates through the filtered candidates to find the process with the minimum `RemainingBurstTime`.
3. **Preemption:** Since this check occurs at every time unit, if a newly arrived process has a shorter remaining time than the currently running process, the scheduler updates the `shortest_index`, effectively preempting the current task.

We deliberately keep the scheduler selection as a linear scan ($O(N)$). Given that $N \leq 10$ in our workload, scanning the array once per tick requires fewer than ~ 50 ns, which is negligible compared to context switch overhead. While a heap-based priority queue offers better asymptotics for large N , it introduces complex preemption bookkeeping and key-update logic that are error-prone. Our choice therefore balances correctness and

maintainability: the linear scan ensures predictable behavior, minimizes race conditions, and avoids unnecessary complexity for small-scale scheduling.

1.3 Multithreading and Synchronization Strategy

To simulate a realistic OS environment where processes run independently, we utilized the pthread library. The system employs a "Scheduler-Worker" model synchronized via **Mutexes** and **Condition Variables**.

- **Global Mutex (global_mutex):** Ensures mutual exclusion so that only one thread (either the scheduler or a process) accesses shared resources (like the log or time variables) at any given moment.
- **Condition Variables (scheduler_cond & my_cond):**
 - The **Scheduler** signals a specific process's my_cond to grant it CPU time.
 - The **Process Thread** runs for one time unit, decrements its burst time, and then signals scheduler_cond to return control to the scheduler.
 - This "handshake" mechanism ensures precise time-step simulation without race conditions.

Scheduler -> Signal -> Process Thread -> Work 1s -> Signal -> Scheduler

The lock-step model exists to keep every simulated tick deterministic. User-space pthread scheduling cannot guarantee that a thread runs for exactly one second, so reproducing precise burst lengths would be unreliable. Our solution is a strict handshake: the scheduler gives run permission to one process and waits; that process decrements its RemainingBurstTime once, signals the scheduler back, and blocks again. Repeating this handshake per tick yields a perfectly reproducible Gantt chart.

1.4 State Management

The system implements a State Transition Diagram consisting of three primary states:

- **READY:** Processes that have arrived but are waiting for the CPU.
- **RUNNING:** The process currently selected by the search_shortest function. Only one process can be in this state at a time.
- **COMPLETED:** Processes with RemainingBurstTime == 0. These are excluded from future scheduling.

1.5 Input Validation and Robustness

To prevent simulation crashes and ensure logical consistency, rigorous input validation is implemented:

- **Negative Values:** Checks verify that Arrival Time is non-negative and Burst Time is positive.

- **Data Type Checks:** The program handles non-integer inputs (e.g., characters) by clearing the input buffer and prompting the user again, ensuring the scheduler always operates on valid integers.

Code Expect

```
// Helper to read integer input safely
int read_int(const char *prompt, int min, int max) {
    int value;
    char buffer[100];

    while (1) {
        printf("%s", prompt);
        if (fgets(buffer, sizeof(buffer), stdin) != NULL) {
            char *endptr;
            value = strtol(buffer, &endptr, 10);

            if (endptr == buffer || *endptr != '\n') {
                printf("Invalid input. Please enter a number.\n");
            } else if (value < min || value > max) {
                printf("Error: Number must be between %d and %d.\n", min, max);
            } else {
                return value;
            }
        }
    }
}
```

Input handling was rewritten around a 'read_int' helper that uses 'fgets' plus 'strtol', so stray characters or overly long lines can no longer corrupt the simulation. All pthread operations funnel through 'check_pthread', producing descriptive errors and exiting safely when a call fails. Log writes also go through a single 'add_log' function that checks for buffer overflow and aborts cleanly if the configured maximum would be exceeded.

2. Test Cases

Test Case 1 : Preemption occurs frequently

Objective: To verify whether the currently running process is correctly preempted when a shorter process arrives.

Input:

Process 1: Arrival = 0, Burst = 7

Process 2: Arrival = 2, Burst = 4

Process 3: Arrival = 4, Burst = 1

Process 4: Arrival = 5, Burst = 3

Theoretical behavior:

Unit time	Process	Waiting Queue
		P1 = 7
0 ~ 2	P1 = 5	P2 = 4, P1 = 5 (at 2)
2 ~ 4	P2 = 2	P3 = 1, P2 = 2, P1 = 5 (at 4)
4 ~ 5	P3 = 0	P2 = 2, P4 = 3, P1 = 5 (at 5)
5 ~ 7	P2 = 0	P4 = 3, P1 = 5 (at 7)
7 ~ 10	P4 = 0	P1 = 5 (at 10)
10 ~ 15	P5 = 0	-

Actual Output:

Time	Process ID	Status	Remaining Time
0	P1	READY	7
0	P1	RUNNING	7
1	P1	RUNNING	6
2	P2	READY	4
2	P2	RUNNING	4
3	P2	RUNNING	3
4	P3	READY	1
4	P3	RUNNING	1
5	P3	COMPLETED	0
5	P2	RUNNING	2
6	P2	RUNNING	1
7	P2	COMPLETED	0
7	P4	READY	3
7	P4	RUNNING	3
8	P4	RUNNING	2
9	P4	RUNNING	1
10	P4	COMPLETED	0
10	P1	RUNNING	5
11	P1	RUNNING	4
12	P1	RUNNING	3
13	P1	RUNNING	2
14	P1	RUNNING	1
15	P1	COMPLETED	0

--- Visual Gantt Chart ---

	P1		P2		P3		P2		P4		P1	
0		2		4		5		7		10		15

--- SRTF Performance Results ---

Process 1: Turnaround = 15, Waiting = 8, Response = 0
Process 2: Turnaround = 5, Waiting = 1, Response = 0
Process 3: Turnaround = 1, Waiting = 0, Response = 0
Process 4: Turnaround = 5, Waiting = 2, Response = 2

Average Turnaround Time = 6.50
Average Waiting Time = 2.75
Average Response Time = 0.50

Discussion:

In this scenario, the SRTF algorithm's preemptive nature is clearly demonstrated. At $t = 2$ and $t = 4$, a new process arrives with a shorter burst time than the currently running process. The scheduler correctly performs a context switch immediately. This minimizes the waiting time for short processes (P2 and P3), which is the primary goal of SRTF. The comparison between theoretical and actual output confirms that the implementation handles preemption logic correctly.

Test Case 2 : CPU IDLE

Objective: To verify that the CPU will be IDLE state after all the processes executed

Input:

Process 1: Arrival = 0, Burst = 15

Process 2: Arrival = 13, Burst = 1

Process 3: Arrival = 18, Burst = 5

Process 4: Arrival = 19, Burst = 4

Process 5: Arrival = 21, Burst = 1

Theoretical behavior

Unit time	Process	Waiting Queue
		P1 = 15
0 ~ 13	P1 = 2	P2 = 1, P1 = 2
13 ~ 14	P2 = 0	P1 = 2
14 ~ 16	P1 = 0	-
16 ~ 18	IDLE	P3 = 5
18 ~ 21	P3 = 2	P5 = 1, P3 = 2, P4 = 4
21 ~ 22	P5 = 0	P3 = 2, P4 = 4
22 ~ 24	P3 = 0	P4 = 4
24 ~ 28	P4 = 0	-

Actual Output

Time	Process ID	Status	Remaining Time
0	P1	READY	15
0	P1	RUNNING	15
1	P1	RUNNING	14
2	P1	RUNNING	13
3	P1	RUNNING	12
4	P1	RUNNING	11
5	P1	RUNNING	10
6	P1	RUNNING	9
7	P1	RUNNING	8
8	P1	RUNNING	7
9	P1	RUNNING	6
10	P1	RUNNING	5
11	P1	RUNNING	4
12	P1	RUNNING	3
13	P2	READY	1
13	P2	RUNNING	1
14	P2	COMPLETED	0
14	P1	RUNNING	2
15	P1	RUNNING	1
16	P1	COMPLETED	0
16	---	IDLE	---
17	---	IDLE	---
18	P3	READY	5
18	P3	RUNNING	5
19	P3	RUNNING	4
20	P3	RUNNING	3
21	P5	READY	1
21	P5	RUNNING	1
22	P5	COMPLETED	0
22	P3	RUNNING	2
23	P3	RUNNING	1
24	P3	COMPLETED	0
24	P4	READY	4
24	P4	RUNNING	4
25	P4	RUNNING	3
26	P4	RUNNING	2
27	P4	RUNNING	1
28	P4	COMPLETED	0

--- Visual Gantt Chart ---

P1	P2	P1	IDLE	P3	P5	P3	P4
0	13	14	16	18	21	22	24
							28

--- SRTF Performance Results ---

Process 1: Turnaround = 16, Waiting = 1, Response = 0
Process 2: Turnaround = 1, Waiting = 0, Response = 0
Process 3: Turnaround = 6, Waiting = 1, Response = 0
Process 4: Turnaround = 9, Waiting = 5, Response = 5
Process 5: Turnaround = 1, Waiting = 0, Response = 0

Average Turnaround Time = 6.60
Average Waiting Time = 1.40
Average Response Time = 1.00

Discussion:

This scenario clearly illustrates the behaviour when all processes in the WaitingQueue have been processed by the SRTF algorithm.

At $t = 16$, once all processes within the WaitingQueue have completed execution, the CPU transitions to the IDLE state.

Subsequently, the moment a new Process arrives at the WaitingQueue, it is recognised as the next process to be processed, and processing commences.

Test Case 3 : Long waiting queue

Objective: To verify that if there is a long waiting queue, the scheduler can select correct process with the shortest remaining burst time

Input:

Process 1: Arrival = 0, Burst = 3

Process 2: Arrival = 1, Burst = 1

Process 3: Arrival = 5, Burst = 20

Process 4: Arrival = 8, Burst = 12

Process 5: Arrival = 10, Burst = 9

Process 6: Arrival = 13, Burst = 13

Process 7: Arrival = 20, Burst = 11

Process 8: Arrival = 21, Burst = 3

Process 9: Arrival = 28, Burst = 5

Process 10: Arrival = 29, Burst = 1

Theoretical behavior

Unit time	Process	Waiting Queue
		P1 = 3
0 ~ 1	P1 = 2	P2 = 1, P1 = 2
1 ~ 2	P2 = 0	P1 = 2
2 ~ 4	P1 = 0	-
4 ~ 5	IDLE	P3 = 20 (at 5)
5 ~ 8	P3 = 17	P4 = 12, P3 = 17
8 ~ 10	P4 = 10	P5 = 9, P4 = 10, P3 = 17
10 ~ 13	P5 = 6	P6 = 3, P5 = 6, P4 = 10, P3 = 17
13 ~ 16	P6 = 0	P5 = 6, P4 = 10, P3 = 17
16 ~ 21	P5 = 1	P5 = 1, P8 = 3, P4 = 10, P7 = 11, P3 = 17
21 ~ 22	P5 = 0	P8 = 3, P4 = 10, P7 = 11, P3 = 17
22 ~ 25	P8 = 0	P4 = 10, P7 = 11, P3 = 17
25 ~ 28	P4 = 7	P9 = 5, P4 = 7, P7 = 11, P3 = 17

28 ~ 29	P9 = 4	P10 = 1, P9 = 4, P4 = 7, P7 = 11, P3 = 17
29 ~ 30	P10 = 0	P9 = 4, P4 = 7, P7 = 11, P3 = 17
30 ~ 34	P9 = 0	P4 = 7, P7 = 11, P3 = 17
34 ~ 41	P4 = 0	P7 = 11, P3 = 17
41 ~ 52	P7 = 0	P3 = 17
52 ~ 69	P3 = 0	-

Actual Output

Time	Process ID	Status	Remaining Time
0	P1	READY	3
0	P1	RUNNING	3
1	P2	READY	1
1	P2	RUNNING	1
2	P2	COMPLETED	0
2	P1	RUNNING	2
3	P1	RUNNING	1
4	P1	COMPLETED	0
4	---	IDLE	---
5	P3	READY	20
5	P3	RUNNING	20
6	P3	RUNNING	19
7	P3	RUNNING	18
8	P4	READY	12
8	P4	RUNNING	12
9	P4	RUNNING	11
10	P5	READY	9
10	P5	RUNNING	9
11	P5	RUNNING	8
12	P5	RUNNING	7
13	P6	READY	3
13	P6	RUNNING	3
14	P6	RUNNING	2
15	P6	RUNNING	1
16	P6	COMPLETED	0
16	P5	RUNNING	6
17	P5	RUNNING	5
18	P5	RUNNING	4
19	P5	RUNNING	3
20	P5	RUNNING	2
21	P5	RUNNING	1
22	P5	COMPLETED	0
22	P8	READY	3
22	P8	RUNNING	3
23	P8	RUNNING	2
24	P8	RUNNING	1
25	P8	COMPLETED	0
25	P4	RUNNING	10
26	P4	RUNNING	9
27	P4	RUNNING	8
28	P9	READY	5
28	P9	RUNNING	5
29	P10	READY	1
29	P10	RUNNING	1
30	P10	COMPLETED	0
30	P9	RUNNING	4
31	P9	RUNNING	3
32	P9	RUNNING	2
34	P4	RUNNING	7
35	P4	RUNNING	6
36	P4	RUNNING	5
37	P4	RUNNING	4
38	P4	RUNNING	3
39	P4	RUNNING	2
40	P4	RUNNING	1
41	P4	COMPLETED	0
41	P7	READY	11
41	P7	RUNNING	11
42	P7	RUNNING	10
43	P7	RUNNING	9
44	P7	RUNNING	8
45	P7	RUNNING	7
46	P7	RUNNING	6
47	P7	RUNNING	5

48	P7	RUNNING	4
49	P7	RUNNING	3
50	P7	RUNNING	2
51	P7	RUNNING	1
52	P7	COMPLETED	0
52	P3	RUNNING	17
53	P3	RUNNING	16
54	P3	RUNNING	15
55	P3	RUNNING	14
56	P3	RUNNING	13
57	P3	RUNNING	12
58	P3	RUNNING	11
59	P3	RUNNING	10
60	P3	RUNNING	9
61	P3	RUNNING	8
62	P3	RUNNING	7
63	P3	RUNNING	6
64	P3	RUNNING	5
65	P3	RUNNING	4
66	P3	RUNNING	3
67	P3	RUNNING	2
68	P3	RUNNING	1
69	P3	COMPLETED	0

--- Visual Gantt Chart ---

P1	P2	P1	IDLE	P3	P4	P5	P6	P5	P8	P4	P9	P10	P9	P4	P7	P3	
0	1	2	4	5	8	10	13	16	22	25	28	29	30	34	41	52	69

--- SRTF Performance Results ---

Process 1: Turnaround = 4, Waiting = 1, Response = 0
Process 2: Turnaround = 1, Waiting = 0, Response = 0
Process 3: Turnaround = 64, Waiting = 44, Response = 0
Process 4: Turnaround = 33, Waiting = 21, Response = 0
Process 5: Turnaround = 12, Waiting = 3, Response = 0
Process 6: Turnaround = 3, Waiting = 0, Response = 0
Process 7: Turnaround = 32, Waiting = 21, Response = 21
Process 8: Turnaround = 4, Waiting = 1, Response = 1
Process 9: Turnaround = 6, Waiting = 1, Response = 0
Process 10: Turnaround = 1, Waiting = 0, Response = 0

Average Turnaround Time = 16.00
Average Waiting Time = 9.20
Average Response Time = 2.20

Discussion:

This scenario clearly demonstrates the scheduler's behaviour when processes are inserted into the WaitingQueue at short intervals after the CPU transitions to the IDLE state following the queue emptying.

At $t = 4$, the CPU transitions to the IDLE state once, after which processes with long BT and then short BT arrive in succession, being processed with precise preemption.

Test case 4 : Simultaneous Arrival

Cases where multiple processes arrive at Time 0 or the same time. Shows how priority is determined by ID order or programme logic.

Objective: To verify that the scheduler will select the process with the shortest remaining burst time if there are multiple process arrived at the same timing

Input:

Process 1: Arrival = 0, Burst = 6

Process 2: Arrival = 0, Burst = 5

Process 3: Arrival = 0, Burst = 2

Process 4: Arrival = 2, Burst = 3

Process 5: Arrival = 2, Burst = 1

Process 6: Arrival = 2, Burst = 2

Theoretical behavior

Unit time	Process	Waiting Queue
		P3 = 2, P2 = 5, P1 = 6
0 ~ 2	P3 = 0	P5 = 1, P6 = 2, P4 = 3, P2 = 5, P1 = 6
2 ~ 3	P5 = 0	P6 = 2, P4 = 3, P2 = 5, P1 = 6
3 ~ 5	P6 = 0	P4 = 3, P2 = 5, P1 = 6
5 ~ 8	P4 = 0	P2 = 5, P1 = 6
8 ~ 13	P2 = 0	P1 = 6

13 ~ 19	P1 = 0	-
---------	--------	---

Actual Output

Time	Process ID	Status	Remaining Time
0	P3	READY	2
0	P3	RUNNING	2
1	P3	RUNNING	1
2	P3	COMPLETED	0
2	P5	READY	1
2	P5	RUNNING	1
3	P5	COMPLETED	0
3	P6	READY	2
3	P6	RUNNING	2
4	P6	RUNNING	1
5	P6	COMPLETED	0
5	P4	READY	3
5	P4	RUNNING	3
6	P4	RUNNING	2
7	P4	RUNNING	1
8	P2	READY	5
8	P2	RUNNING	5
9	P2	RUNNING	4
10	P2	RUNNING	3
11	P2	RUNNING	2
12	P2	RUNNING	1
13	P2	COMPLETED	0
13	P1	READY	6
13	P1	RUNNING	6
14	P1	RUNNING	5
15	P1	RUNNING	4
16	P1	RUNNING	3
17	P1	RUNNING	2
18	P1	RUNNING	1
19	P1	COMPLETED	0

--- Visual Gantt Chart ---

P3	P5	P6	P4	P2	P1	
0	2	3	5	8	13	19

--- SRTF Performance Results ---

Process 1: Turnaround = 19, Waiting = 13, Response = 13
Process 2: Turnaround = 13, Waiting = 8, Response = 8
Process 3: Turnaround = 2, Waiting = 0, Response = 0
Process 4: Turnaround = 6, Waiting = 3, Response = 3
Process 5: Turnaround = 1, Waiting = 0, Response = 0
Process 6: Turnaround = 3, Waiting = 1, Response = 1

Average Turnaround Time = 7.33
Average Waiting Time = 4.17
Average Response Time = 4.17

Discussion:

This scenario clearly illustrates the scheduler's behaviour when multiple processes arrive simultaneously at a given point in time.

First, at $t = 0$, three processes arrive with distinct BurstTimes. At this point, the processes enter the READY state within the WaitingQueue. In this project's code, implementing a WaitingQueue is anticipated to incur spatial overhead, so it is not implemented. Instead, the implementation involves sequentially searching the process queue containing all processes to find and execute the process with the shortest RemainingBurstTime. The algorithm for finding the process with the shortest RBT searches for the process with an RBT 'less than' the current minimum. Therefore, if multiple processes with the same RBT arrive simultaneously, they will be executed in the order of their arrival IDs.

In this test case, all processes arrived with different BurstTimes, so they were executed in order from the shortest.

However, at $t = 2$, a process with a shorter BurstTime than the initial three processes arrived, so it was correctly preempted and processed.

Test Case 5 : Confirming long job starvation

A scenario where short processes arrive intermittently, preventing long processes (e.g., burst 100) from executing.

Objective: To verify the behaviour of long job in the scheduler

Input:

Process 1: Arrival = 0, Burst = 100

Process 2: Arrival = 0, Burst = 10

Process 3: Arrival = 3, Burst = 7

Process 4: Arrival = 16, Burst = 8

Process 5: Arrival = 25, Burst = 12

Process 6: Arrival = 35, Burst = 5

Theoretical behavior:

Unit time	Process	Waiting Queue
		P2 = 10, P1 = 100
0 ~ 3	P2 = 7	P2 = 7, P3 = 7, P1 = 100
3 ~ 10	P2 = 0	P3 = 7, P1 = 100
10 ~ 16	P3 = 1	P3 = 1, P4 = 8, P1 = 100
16 ~ 17	P3 = 0	P4 = 8, P1 = 100
17 ~ 25	P4 = 0	P5 = 12, P1 = 100
25 ~ 35	P5 = 2	P5 = 2, P6 = 5, P1 = 100
35 ~ 37	P5 = 0	P6 = 5, P1 = 100
37 ~ 42	P6 = 0	P1 = 100
42 ~ 142	P1 = 0	-

Actual Output:

Time	Process ID	Status	Remaining Time
0	P2	READY	10
0	P2	RUNNING	10
1	P2	RUNNING	9
2	P2	RUNNING	8
3	P2	RUNNING	7
4	P2	RUNNING	6
5	P2	RUNNING	5
6	P2	RUNNING	4
7	P2	RUNNING	3
8	P2	RUNNING	2
9	P2	RUNNING	1
10	P2	COMPLETED	0
10	P3	READY	7
10	P3	RUNNING	7
11	P3	RUNNING	6
12	P3	RUNNING	5
13	P3	RUNNING	4
14	P3	RUNNING	3
15	P3	RUNNING	2
16	P3	RUNNING	1
17	P3	COMPLETED	0
17	P4	READY	8
17	P4	RUNNING	8
18	P4	RUNNING	7
19	P4	RUNNING	6
20	P4	RUNNING	5
21	P4	RUNNING	4
22	P4	RUNNING	3
23	P4	RUNNING	2
24	P4	RUNNING	1
25	P4	COMPLETED	0
25	P5	READY	12
25	P5	RUNNING	12
26	P5	RUNNING	11
27	P5	RUNNING	10
28	P5	RUNNING	9
29	P5	RUNNING	8
30	P5	RUNNING	7
31	P5	RUNNING	6
32	P5	RUNNING	5
33	P5	RUNNING	4
34	P5	RUNNING	3
35	P5	RUNNING	2
36	P5	RUNNING	1
37	P5	COMPLETED	0
37	P6	READY	5
37	P6	RUNNING	5
38	P6	RUNNING	4
39	P6	RUNNING	3

40	P6	RUNNING	2
41	P6	RUNNING	1
42	P6	COMPLETED	0
42	P1	READY	100
42	P1	RUNNING	100
43	P1	RUNNING	99
44	P1	RUNNING	98
45	P1	RUNNING	97
46	P1	RUNNING	96
47	P1	RUNNING	95
48	P1	RUNNING	94
49	P1	RUNNING	93
50	P1	RUNNING	92
51	P1	RUNNING	91
52	P1	RUNNING	90
53	P1	RUNNING	89
54	P1	RUNNING	88
55	P1	RUNNING	87
56	P1	RUNNING	86
57	P1	RUNNING	85
58	P1	RUNNING	84
59	P1	RUNNING	83
60	P1	RUNNING	82
61	P1	RUNNING	81
62	P1	RUNNING	80
63	P1	RUNNING	79
64	P1	RUNNING	78
65	P1	RUNNING	77
66	P1	RUNNING	76
67	P1	RUNNING	75
68	P1	RUNNING	74
69	P1	RUNNING	73
70	P1	RUNNING	72
71	P1	RUNNING	71
72	P1	RUNNING	70
73	P1	RUNNING	69
74	P1	RUNNING	68
75	P1	RUNNING	67
76	P1	RUNNING	66
77	P1	RUNNING	65
78	P1	RUNNING	64
79	P1	RUNNING	63
80	P1	RUNNING	62
81	P1	RUNNING	61
82	P1	RUNNING	60

112	P1	RUNNING	30
113	P1	RUNNING	29
114	P1	RUNNING	28
115	P1	RUNNING	27
116	P1	RUNNING	26
117	P1	RUNNING	25
118	P1	RUNNING	24
119	P1	RUNNING	23
120	P1	RUNNING	22
121	P1	RUNNING	21
122	P1	RUNNING	20
123	P1	RUNNING	19
124	P1	RUNNING	18
125	P1	RUNNING	17
126	P1	RUNNING	16
127	P1	RUNNING	15
128	P1	RUNNING	14
129	P1	RUNNING	13
130	P1	RUNNING	12
131	P1	RUNNING	11
132	P1	RUNNING	10
133	P1	RUNNING	9
134	P1	RUNNING	8
135	P1	RUNNING	7
136	P1	RUNNING	6
137	P1	RUNNING	5
138	P1	RUNNING	4
139	P1	RUNNING	3
140	P1	RUNNING	2
141	P1	RUNNING	1
142	P1	COMPLETED	0

--- Visual Gantt Chart ---

P2	P3	P4	P5	P6	P1	
0	10	17	25	37	42	142

--- SRTF Performance Results ---

Process 1: Turnaround = 142, Waiting = 42, Response = 42

Process 2: Turnaround = 10, Waiting = 0, Response = 0

Process 3: Turnaround = 14, Waiting = 7, Response = 7

Process 4: Turnaround = 9, Waiting = 1, Response = 1

Process 5: Turnaround = 12, Waiting = 0, Response = 0

Process 6: Turnaround = 7, Waiting = 2, Response = 2

Average Turnaround Time = 32.33

Average Waiting Time = 8.67

Average Response Time = 8.67

Discussion:

This scenario demonstrates that when a process with a long time-to-run arrives first, followed by processes with shorter time-to-run arriving in sequence, the initial process with the long time-to-run remains perpetually unable to execute, resulting in a so-called “starvation” state.

As this code does not incorporate ageing, the process with the long time-to-run will never execute as long as processes with shorter time-to-run continue to arrive.

Input Validation

```

Enter number of processes (1-10): o
Invalid input. Please enter a number (1-10): -1
Error: Number must be between 1 and 10. Enter again: 4
Process 1
Arrival (non-negative number): p
Invalid input. Please enter a number: -5
Error: Arrival time cannot be negative. Enter again: 4
Burst (non-negative number): f
Invalid input. Please enter a number: -3
Error: Burst time must be greater than 0. Enter again: 9

```

Additional supporting software code (if there is any) or the explanation about the software features used in the program

3. Discussion

3.1 Quantitative Comparison between Theory and Simulation

The simulation results demonstrated absolute consistency with the theoretical calculations across all test cases, verifying the correctness of the implemented `search_shortest` algorithm and state transition logic.

- Accuracy of Preemption Timing (Test Case 1):**
 In Test Case 1, the theoretical model predicted that Process 1 (P1) would be preempted by Process 2 (P2) at time $t=2$, and P2 would subsequently be preempted by Process 3 (P3) at $t=4$. The generated Gantt chart confirms these exact transition points: P1 yields the CPU at $t=2$ and P2 yields at $t=4$. Furthermore, the calculated Average Waiting Time in the simulation was 2.75 units, which matches the theoretical value of 2.75 exactly. This proves that the scheduler correctly evaluates the "Remaining Burst Time" at every time unit.
- Handling of Simultaneous Arrivals (Test Case 4):**
 Test Case 4 validated the tie-breaking logic. When P1, P2, P3 arrived simultaneously at $t=0$, the scheduler correctly selected P3 (Burst=2) first. The resulting Average Turnaround Time of 7.33 units aligns with the manual trace, confirming that the priority logic based on burst time is functioning without error.

3.2 Impact of Process Arrival Variation on Efficiency

The simulation revealed a significant trade-off between Responsiveness and Resource Efficiency depending on the density of process arrivals.

- **Sparse Arrivals and CPU Efficiency** (Analysis of Test Case 3):
In Test Case 3, widely spaced arrival times led to CPU underutilization. The simulation log shows the CPU entering the IDLE state at intervals such as $t=4\sim5$ and $t=16\sim21$. While SRTF provides excellent response times for individual processes in this scenario, the overall System Throughput is reduced due to these idle periods. This suggests that in a real-world batch system, a long-term scheduler might be needed to fill these gaps.
- **Dense Arrivals and Overhead** (Analysis of Test Case 5):
Test Case 5 represented a "dense" arrival scenario where short processes (P2 to P6) arrived frequently while a long process (P1) was executing.
 1. **Starvation:** P1 (Burst=100) started at $t=0$ but was repeatedly preempted and could not complete until $t=142$, resulting in a high Turnaround Time of 142. This quantitatively demonstrates the Starvation problem inherent in SRTF without an aging mechanism.
 2. **Context Switching Overhead:** The simulation performed context switches frequently (e.g., at $t=3, 10, 16, 25, 35$). While this simulation assumes zero-cost switching, in a real operating system, each switch requires saving the PCB, flushing the TLB/cache, and loading the new state. In such high-frequency preemption scenarios (as seen in Test 5), this **overhead** would degrade actual CPU performance compared to non-preemptive algorithms like SJF.

4. Conclusion

In this project, we constructed a simulation environment for the SRTF scheduling algorithm using the C language and the pthread library. The main implementation achievements are as follows.

1. **Dynamic priority management:** Correct preemption behaviour was achieved by implementing logic that constantly monitors `RemainingBurstTime` via the `search\_shortest` function and selects the process with the minimum remaining time.
2. **Multithreaded synchronisation:** By employing condition variables, precise synchronisation of execution control between the scheduler and each process (thread) was achieved.
3. **Input robustness:** Validation functionality has been implemented to exclude invalid inputs such as negative values or characters.

On the other hand, the experiment also confirmed the issue of "long-running process starvation" within SRTF. In real-world operating systems, "Aging" technology is employed alongside this to address the problem, which increases a process's priority based on its waiting time (like HRRN scheduling). Future prospects include adding this Aging functionality and extending the simulation to be more realistic by incorporating the overhead time associated with context switches.

5. Appendix Source code listing (with comments for readability)

[SRTF-Simulation/MultiThreading.c at main · ibukye/SRTF-Simulation · GitHub](#)

6. AI Tools and External Sources Usage

Generative AI tools (ChatGPT) were used to get the idea of how to implement multithreading and debug that logic and refine the grammar and structure of this report.

The core algorithm design and coding were performed by the group members.

The pthread implementation references were consulted from

- Title: 【C言語】 マルチスレッドの使い方 [How to Use Multithreading in C] (pokoroblog)
URL: <https://pokoroblog.com/%E3%80%90c%E8%A8%80%E8%AA%9E%E3%80%91%E3%83%9E%E3%83%AB%E3%83%81%E3%82%B9%E3%83%AC%E3%83%83%E3%83%89%E3%81%AE%E4%BD%BF%E3%81%84%E6%96%B9/> (Accessed: Nov 5, 2025)
- Title: C言語でスレッド間同期 [Thread Synchronization in C] (Qiita)
URL: https://qiita.com/miyamoto_works/items/8ede26b07099606e50db (Accessed: Nov 5, 2025)
- Title: [C言語] マルチスレッドを触ってみる [Exploring Multithreading in C] (zenn)
URL: <https://zenn.dev/rinngo0302/articles/d1aa031137de9f> (Accessed: Nov 6, 2025)

7. Peer Review form

Member	Ibuki Furusho (20716304)	Kazuki Ichikawa ()	Jeffrey Teoh Dass ()
Single Thread	x	x	
Multi Thread	x	x	
Report	x	x	

